

Exception handling:

What is an Exception: Exceptions are raised when the program is syntactically correct, but the code resulted in error. This error does not stop the execution of the program, however it changes the normal flow of the program.

Summary: Exceptions-> Code syntax is right but result in error->Execution continues->Changes the normal flow of program.

Various keyword to handle exceptions:

1.try: The try block lets you test a block of code for errors.

2.except: The except block lets you handle the errors.

Ex:

```
i=int(input("Enter the number:"))
result=100/i
try:
    i=int(input("Enter the number:"))
    result=100/i
except ZeroDivisionError:
    print("Can't divide by zero.")
```

3.else: The else block lets you execute code when there is no errors.

Ex:

```
try:
    i=int(input("Enter a number:"))
    result=250/i
except ZeroDivisionError:
    print("Can't divide by error.")
else:
    print("Division is ",result)
```

4.finally: The finally block lets you execute code, regardless of the result of the try- and except blocks.

Ex:

```
try:
    i=int(input("Enter a number:"))
    result=250/i
```

```
except ZeroDivisionError:
    print("Can't divide by error.")
else:
    print("Division is ",result)
finally:
    print("Division by zero is zero only!")
```

5.raise: The raise keyword is used to throw an exception “if” a condition occurs.

Ex:

```
x = "hello"
if not type(x) is int:
    raise TypeError("Only integers are allowed")
else:
    print("Number is ",x)
```

Ex:

```
x = -1
if (x < 0):
    raise Exception("Sorry, no numbers below zero")
else:
    print("Given number is positive.")
```

6.catch: Try and except statements are used to catch and handle exceptions in Python. Statements that can raise exceptions are kept inside the try clause and the statements that handle the exception are written inside except clause.

Ex:

```
try:
    i=int(input("Enter a number:"))
    result=250/i
except ZeroDivisionError:
    print("Can't divide by error.")
except ValueError:
    print("Enter a valid number you dumb!")
else:
    print("Division is ",result)
```

Network: In Python, networking refers to the use of Python to write scripts and perform tasks on a network. Python is a popular choice for network scripting because of its ease of use and extensive libraries.

Network Connectivity:

Protocol: It is a set of rules.

Types of Protocol: 1.TCP=Transmission Control Protocol (Connection oriented), 2.UDP=User Datagram Protocol (Connectionless)

Difference between TCP and UDP.

TCP Transmission Control Protocol.	UDP User Datagram Protocol.
1.Connection type:	
TCP requires an establish connection before transmitting the data	UDP required no connection to start and end a data transfer.
2.Data sequence:	
It is sent in specific order.	It is not in sequence or we can't arrange it.
3.Data re-transmission:	
Can re-transmit data if packet fails to arrive.	We can't re-transmit data i.e no data can be retrieved.
4.Check for error:	
There is proper error checking, guarantees the data arrival in its intended state.	Minimum error checking, but may not prevent all errors.
5.Broadcasting:	
It is not supported for TCP.	It is supported for UDP.
6.Speed:	
It is time consuming then the UDP.	It is faster then the TCP.

Socket Module: The socket library in Python is a built-in module that allows users to create, manage, and work with sockets:

Sockets: A low-level interface for communication between processes on different or the same machines. Sockets are often used in client-server communication.

Functions: The socket module includes functions for creating sockets, connecting them to addresses, sending and receiving data, and handling errors.

Network protocols: Sockets allow users to establish network connections over various network protocols such as TCP or UDP.

Server Socket Method:

1.socket.bind(): This method binds the address [hostname, port no.] to the socket.

2.socket.listen(): This method listen to the connection made to the socket. It starts as TCP listener.

3.socket.accept(): This will accept TCP client connection.

Protocol, Port no., Python library, Function:

Protocol	Port No.	Library	Function
http: Hypertext Transfer Protocol	80	httplib, urllib	Web pages
ftp: File Transfer Protocol	20	ftplib, urllib	File Transfer
smtp: Simple Mail Transfer Protocol	25	smtplib	For Sending Files
pop3: Post Office Protocol 3	110	poplib	Fetching E-Mail
telnet: Teletype Network Protocol	23	telnetlib	Command lines

Database connectivity:

Installing MySQL connector:

- 1.Install my MySQL connector using pip.
- 2.Import module.
- 3.Connect to the database using connect().
- 4.Create a cursor object with cursor().
- 5.Execute SQL queries using execute().
- 6.Fetch result with fetchall(), fetchone() and fetchmany().

1.Installation: The connector is installed by using the python package manager pip.

Command/code in command prompt: pip install mysql-connector-python.

Installs latest version of the connection, ensuring compatibility with python and MySQL.

2.Importing module:**Ex:**

```
import mysql.connector

connection=mysql.connector.connect(host="Local
Host",user="your_name",password="your_password",database="company_db")

if connection.is_connected():

    print("Connected to MySQL database")
```

3.cursor() function: Creating a cursor syntax is cursor=connection.cursor(). After establishing the connection, you create an cursor object that allows you to execute SQL statements and retrieve data from the database.

Function and role of cursor:

- 1.Executing SQL queries SELECT, UPDATE, INSERT, DELETE.
- 2.Fetching data.

Needs of cursor:

- 1.Execute SQL command in PYTHON.
- 2.Retrieve data from the database for use in application.
- 3.Manage large result efficiently.
- 4.Securely execute queries with user input (Avoiding SQL injection).

4.execute(): Allows you to run SQL statements such as SELECT, DELETE, INSERT, UPDATE. The cursor object uses this method to interact with database.

5.Executing queries: SQL statement are passed as strings to the execute() method.

Ex:

```
Cursor.execute("SELECT * FROM employees WHERE age>20 ")
```

6.Parameterised Queries: To prevent SQL injections parameterised queries are used where placeholders(%) replace actual values, making the code more secure.

Ex:

```
sql=("INERT INTO employees(name,age,department) values(%s,%s,%s)")

values=("Ali",28,"HR")

cursor.execute(sql.values)
```

CRUD(Creation Retrieve Update Deletion) operations: The execute() method can dynamically execute different types of SQL queries.

1.SELECT: To select from a table in MySQL, use the "SELECT" statement.

Ex:

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    password="yourpassword",
    database="mydatabase"
)
cursor = mydb.cursor()
cursor.execute("SELECT name,department FROM employees where age >24")
```

2.INSERT: To fill a table in MySQL, use the "INSERT INTO" statement.

Ex:

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    password="yourpassword",
    database="mydatabase"
)
cursor = mydb.cursor()
cursor.execute(INSERT INTO employees(name,age,department)
VALUES(%s,%s,%s),("John",29,"IT"))
```

3.UPDATE: You can update existing records in a table by using the "UPDATE" statement.

Ex:

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    password="yourpassword",
    database="mydatabase"
)
cursor = mydb.cursor()
cursor.execute("UPDATE employees SET department='Finance' WHERE id=4")
```

4.DELETE: You can delete records from an existing table by using the "DELETE FROM" statement.

Ex:

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    password="yourpassword",
    database="mydatabase"
)
cursor = mydb.cursor()
cursor.execute("DELETE FROM employees WHERE id=4")
```

Types of errors or Built-in Exceptions:

Exceptions or Built in exceptions.	Descriptions.
1. ArithmeticError	Raised when an error occurs in numeric calculations.
2. AssertionError	Raised when an assert statement fails.
3. FloatingPointError	Raised when a floating point calculation fails.
4. ImportError	Raised when an imported module does not exist.
5. IndentationError	Raised when indentation is not correct.
6. IndexError	Raised when an index of sequence does not exist.
7. KeyError	Raised when a key does not exist in dictionary.
8. KeyboardInterrupt	Raised when a user presses ctrl+c, ctrl+z and delete.
9. NameError	Raised when a variable does not exist.
10. ReferenceError	Raised when a weak reference object does not exist.
11. RuntimeError	Raised when an error occurs that do not belong to any specific exceptions.
12. SyntaxError	Raised when a syntax error occurs.
13. TypeError	Raised when two different types are combined.
14. SystemError	Raised when a system error occurs.
15. ZeroDivisionError	Raised when the second operator in a division is zero.
16. ValueError	Raised when there is a wrong value in a specified data type.
17. FileNotFoundError	Raised when an attempt is made to access a file that does not exist.
18. AttributeError	Raised when attribute reference or assignment fails.
19. IOError	Raised when an input/output operation fails, such as when an attempt is made to open a non-existent file.